Session 6.1

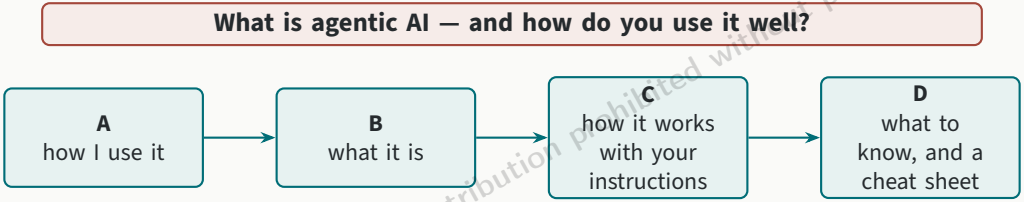
Agentic AI: How I Use It, What It Is, How It Works, and How to Use It Well



© Zhigang Feng Redistribution prohibited without permission

Zhigang Feng

The Question, and the Route



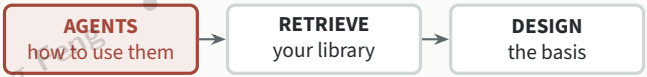
Redistribution prohibited without permission
© Zhigang Feng

- Session 5 ended with a model that gives advice. This one acts: it runs code, moves files, drafts.
- Four parts: how I use it, what it is, how it works with your instructions, what to know to use it well.
- **You stay the project manager. The agent does the typing.**

Session 6.1

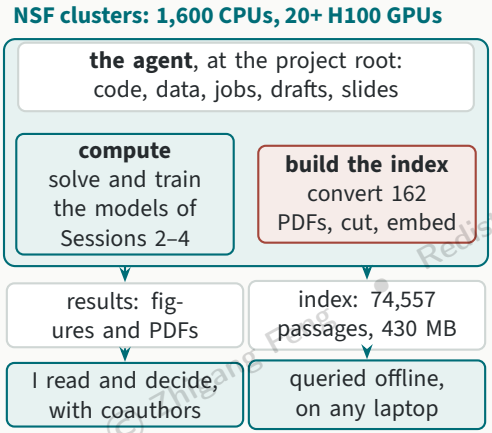
How I Use It

An overview: a cluster, a library, one method



© Zhigang Feng Redistribution prohibited without permission

How I Work: Everything on a Cluster, and a Library the Agent Can Read



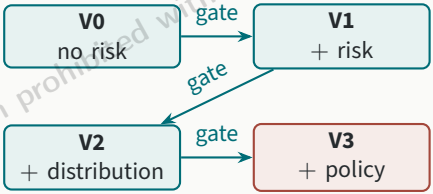
- The agent runs where the code, data and jobs are: a login node. What leaves the cluster is a PDF I read.
- It also answers from my own library, citing the page, or says the evidence is weak: **RAG**, Deck 6.2.
- **Build where the data and the GPUs are; read and decide anywhere.**

© Ameen Reza ? Redistribution prohibited without permission

Source: The instructor's retrieval system and project record, read 12 October 2026; allocations through NSF grants. Deck 6.2 builds the index.

One Method: Solve the Easy Version First

- Start from a version you understand, with a benchmark you trust.
- Add one margin at a time; a new bug belongs to the margin you just added.
- **Never ask the agent for the full problem in one jump.**



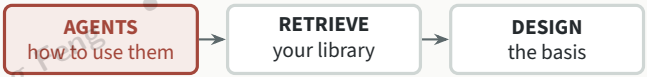
homotopy: deform a solved problem toward the target, one validated step at a time; a failed step returns to the last version that passed

© Zhigang Feng
Redistribution prohibited without permission

Session 6.1

What It Is

Chatbot, copilot, agent, team — and what makes AI agentic



© Zhigang Feng Redistribution prohibited without permission

From a Chat Box to Agent Teams, 2022–2026



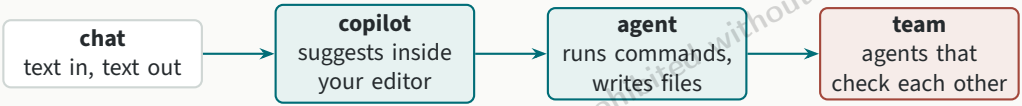
Redistribution prohibited without permission

© Zhaonan Zhang

- Each year the model could touch more of your machine: answer, call tools, share plumbing, act on files, run unattended.
- **Each step made the work cheaper. None made checking it cheaper.**

Source: Dates checked against vendor announcements, September 2026.

Chat, Copilot, Agent, Team: A Spectrum, Not a Switch



- What changes along the strip is not the model. It is how much of your machine it may touch.
- One measured point: 79% of terminal-agent conversations were automation, against 49% in a chat window.
- **Choose a position per task, and move left when the task is delicate.**

Source: Anthropic Economic Index, “AI’s impact on software development”, 28 Apr 2025 (read 23 Sep 2026).

© Zhigang Feng Redistribution prohibited without permission

What Makes AI Agentic: Four Marks, All Needed

tool use
calls Python, R, Stata, the shell

autonomy
a goal, not a prompt: it plans the steps

iteration
reads the error, fixes, reruns

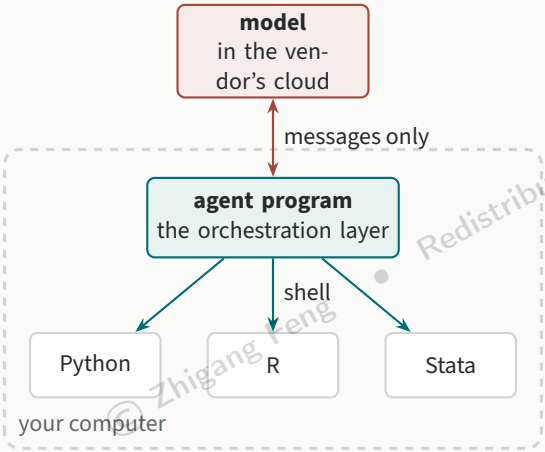
persistence
conventions and plans kept on disk

An **agent** is a model in a loop, given tools and pointed at a goal: the cleverness is in the model, the *capability* in the loop.

- The marks stack: tools without autonomy is a console; autonomy without iteration dies at the first error; nothing survives the session without persistence.
- **"Agentic" is a degree: how much of the loop runs without you.**

© Zhigang Feng
Redistribution prohibited without permission

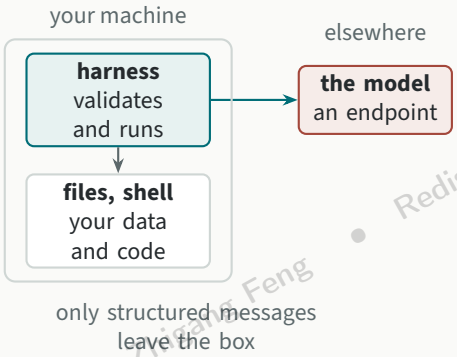
How an Agent Works on Your Machine



- The agent program runs on your machine and does no econometrics itself.
- It asks the model "what next?", then runs that step in the Python, R or Stata you already have.
- **You keep Stata. You gain a layer that types commands, reads errors, and tries again.**

© Zhigang Feng Redistribution prohibited without permission

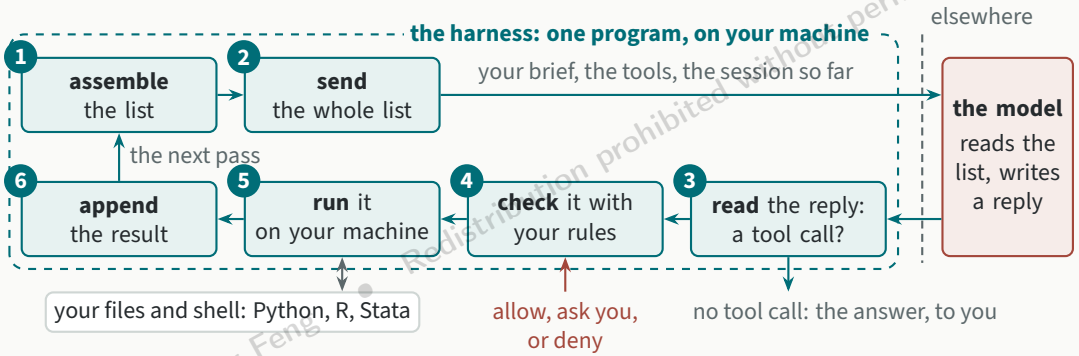
Three Parts, and Only One of Them Is Clever



- **The harness** is an ordinary program on your computer: it runs commands and enforces the rules you set.
- **The model** is an endpoint elsewhere: it reads messages and names the next call. That is the whole of it.
- **The intelligence sits in one part; the discipline in the other two.**

© Zhaoyang Feng
Redistribution prohibited without permission

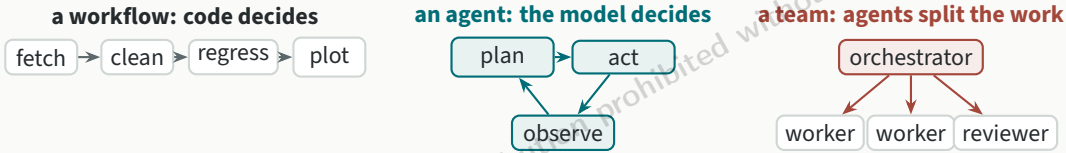
How the Harness Works: Six Steps, Repeated Until No Tool Is Called



© Zhigang Feng Redistribution prohibited without permission

- The model only writes; every read, check and run on your machine is the harness's.
- A refused call comes back as text, like any output; the next pass sends the longer list.
- **Your levers all sit in the harness: the brief, the tools, the rules.**

Agent, Workflow, Team: Who Decides the Next Step



© Zhiqiang Feng Unauthorized distribution prohibited without permission

- Workflow: code decides the steps. Agent: the model decides. Team: agents split the work and check each other.
- Cheap and predictable; handles surprises; buys breadth and pays in coordination. Start at the left.
- **Capability and risk both sit with whoever chooses the next step.**

Where the Agent's Files Live: Your Home Folder and Your Project Folder

your home: every project you open

`~/.claude/`	read for every project
├ `CLAUDE.md`	your standing brief
├ `settings.json`	your rules and hooks
├ `skills/ agents/ rules/`	yours, everywhere
├ `plans/`	plan-mode drafts
└ `projects/<project>/`	
├ `memory/MEMORY.md`	auto memory
└ `<session>.jsonl`	session transcripts
`~/.claude.json`	servers and sign-in

the project: this one, shared with the team

`my-project/`	where you start
├ `CLAUDE.md`	the brief, committed
├ `CLAUDE.local.md`	private, gitignored
├ `.mcp.json`	tool servers, shared
├ `.claude/`	
│ ├ `settings.json`	team rules, committed
│ ├ `settings.local.json`	yours, gitignored
│ ├ `rules/*.md`	rules by topic
│ └ `skills/ agents/`	skills, sub-agents
└ `src/ data/ paper/`	the work itself

© Zhiqiang Zeng · Unauthorized redistribution prohibited without permission

- Same layout, two homes: one for every project you open, one for this project.
- Committed files travel with the repository; `.local` files stay out, on your machine.
- **Both load at every start: briefs add up; settings rank, project above yours.**

Source: Claude Code 2.1.303: documentation on memory, settings, skills and MCP, read 20 October 2026.

Session 6.1

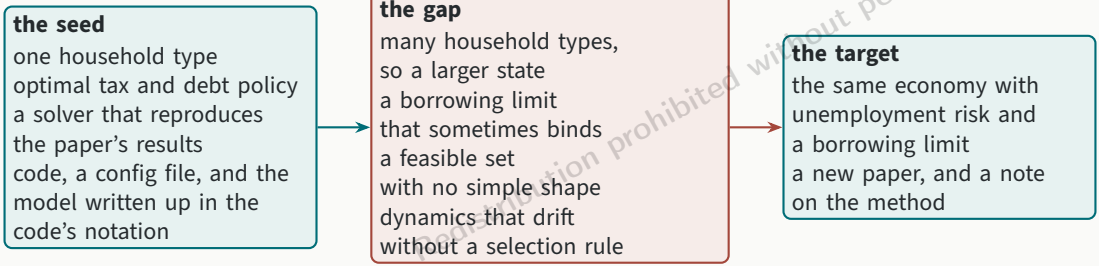
Your Instructions, the Loop, the Workflow

You are the project manager: one project from seed to paper, and how the agent runs each step



© Zhigang Feng Redistribution prohibited without permission

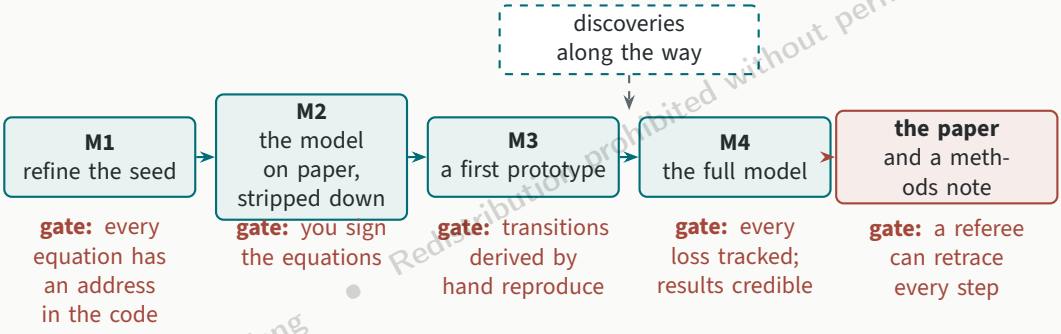
The Project: A Working Seed, a Target, and the Gap Between Them



© Zhiang Feng Reproduction prohibited without permission

- A good seed: it works, it is documented, it is modular, and it points at the target.
- Each item the seed lacks becomes one milestone; two of them became new methods.
- **The plan is the list of what the seed lacks.**

The Plan: Four Milestones, Each With Its Gate



© Zhiyang Feng Redistribution prohibited without permission

- One margin at a time: a new bug belongs to the margin you just added.
- Gates are written first; a failed step returns to the last milestone that passed.
- **A milestone is a validated artefact on disk, not a feeling of progress.**

How the Agent Runs a Step: Your Instruction, a Tool Call in JSON, an Observation

you	Restate the model and the algorithm in your words. Then audit the code against the text.
model →	`{"tool": "read_file", "path": "model.tex"}`
harness ←	the write-up, as text
model →	`{"tool": "search", "pattern": "euler", "in": "src/"}`
harness ←	three matches, with file and line
model →	`{"tool": "shell", "cmd": "rm -r old_runs"}`
harness ←	refused: not on your allow-list
model	no tool call: (7) is in model.py, (9) is not implemented, the score formula differs.

- A tool is a typed function you wrote: a name, a schema for its arguments, a description.
- The model answers with one JSON line; the harness runs it and sends the output back.
- **It stops when the model stops calling tools. Your rules can refuse any call.**

© Zhaoyang Tong Redistribution prohibited without permission

Step 2: The New Model on Paper, Before Any Code

you Add unemployment risk and a borrowing limit; write the new equations. No code yet.

model → `{"tool": "write_file", "path": "model_v2.tex", "text": "..."}`

harness ← written

you read the draft. Two plausible equations are wrong:
a budget with next period's consumption; a transition probability running the wrong way.

you Fix both. Keep utility simple and the risk symmetric.

model → `{"tool": "edit_file", "path": "model_v2.tex", "old": "...", "new": "..."}`

harness ← written

- The draft compiles and reads well; no compiler catches an economic error. You do.
- Simplicity is a design decision: log utility and symmetric risk, chosen to be checkable.
- **Gate: the equations carry your signature before a line of code exists.**

© Zhiang Yang
Redistribution prohibited without permission

Step 3: A First Prototype — Write, Run, Read the Error, Fix

you	Implement model_v2.tex in the seed's module layout. Keep the config file.
model →	`{"tool": "write_file", "path": "src/model.py", "text": "..."}`
model →	`{"tool": "run", "cmd": "python train.py --config config.json"}`
harness ←	after a dozen iterations the loss is NaN
model →	`{"tool": "edit_file", "path": "src/model.py", "new": "clamp c_next"}`
model →	`{"tool": "run", "cmd": "python train.py --config config.json"}`
harness ←	residuals falling; the feasible region is drawn
model →	`{"tool": "run", "cmd": "pytest tests/transitions.py"}`
harness ←	all passed: the forward pass matches the hand-derived transitions

© Zhiqiang Feng ● Redistribution prohibited without permission

- An error message is not a failure; it is the next observation. Read, edit, rerun.
- Same module skeleton as the seed, so every diff stays readable to you.
- **Gate: the forward pass reproduces hand-derived transitions, kept as a test.**

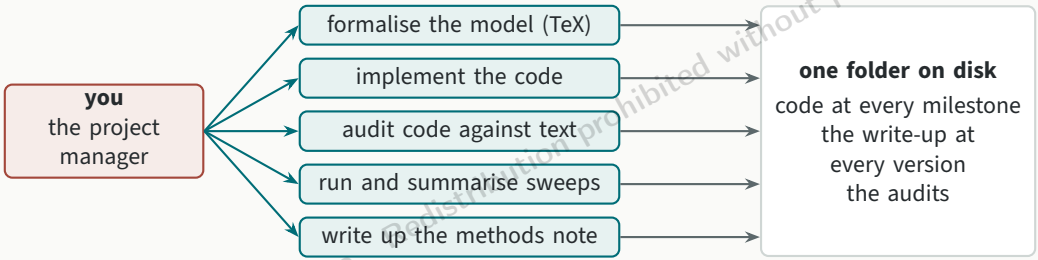
Step 4: The Full Model — Failures Become Structure, Agents Run the Sweeps

you	Paths escape the feasible set; the critic oscillates. Diagnose and propose structure.
model →	`{"tool": "write_file", "path": "PLAN.md", "text": "hypotheses, next runs"}`
model →	`{"tool": "spawn_agent", "task": "run the sweep over config.json; report one table"}`
harness ←	the table, one row per configuration
model →	`{"tool": "spawn_agent", "task": "audit src/ against model_v2.tex"}`
harness ←	two mismatches, with file and line
model	no tool call: project paths back; freeze a target critic; track each loss apart.

- A sub-agent is a second loop with its own window: one runs the sweep, another audits.
- The plan file is memory: it outlives the window and the session.
- **Gate: every loss tracked apart, and a result you find credible.**

© Thiago S. Ferro · Reproduction prohibited without permission

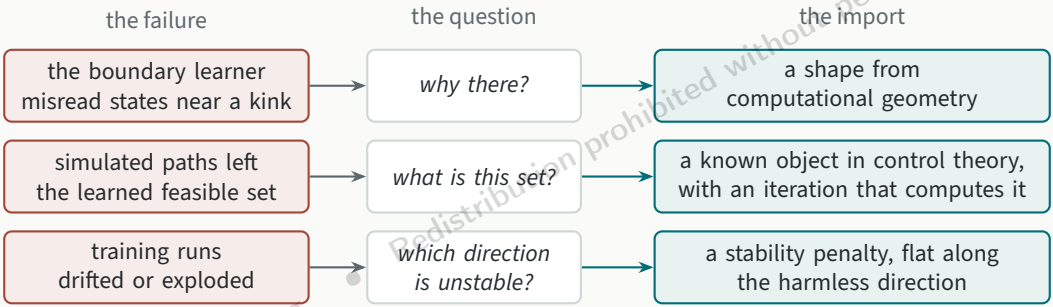
Orchestration: One Manager, Several Sessions, One Folder



© Zhiyang Feng Redistribution prohibited without permission

- One session per layer, each a fresh window; the folder on disk is what they share.
- The reviewer is never the builder: a second session audits what the first one wrote.
- **You integrate. No agent sees the whole project; you do.**

The Creative Part: Three Failures That Became New Methods



© Zhaoming Feng Redistribution prohibited without permission

- Ask why, not for a fix: the agent diagnoses and searches, you name the economics.
- Each import came from a literature economists rarely read, in days rather than years.
- **Three parts of the final algorithm were not in the plan.**

What the Manager Provides, What the Agent Provides

you provide	the agent provides
the seed and the destination	speed between them
economic correctness	syntactic and structural correctness
the gate at each milestone	candidate artefacts to accept or reject
the question “is this credible?”	the diagnostics to answer it

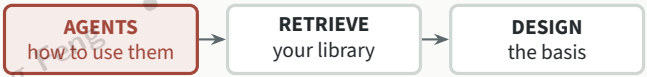
© Zhigang Feng · Redistribution prohibited without permission

- Four things: a working seed, a destination, judgement at the gates, orchestration.
- The agent chose no question, caught no timing error, judged no result credible.
- **The job is the cycle, and the cycle is management.**

Session 6.1

Inside the Machine

What to know to use it well, and one page to take home



© Zhigang Feng Redistribution prohibited without permission

The Session Is a List, Not a Connection

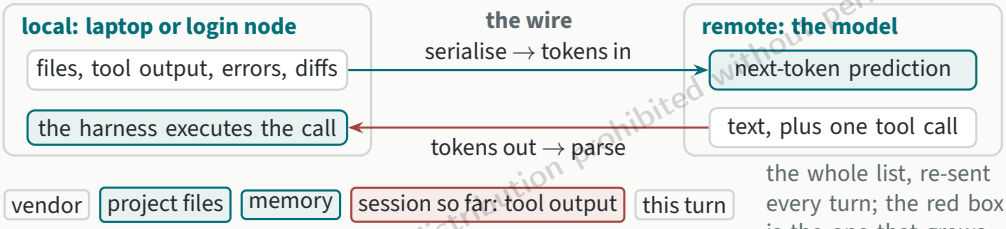


teal: the only new bytes;
white: re-sent word for word,
and paid for again.

- The endpoint keeps nothing. Each turn re-sends the whole conversation so far.
- "The window is full" means the list outgrew the limit, and it grows fastest from tool output, not from what you type.
- **Clearing, compacting and resuming are all operations on a list.**

© Zhigang Peng
Redistribution prohibited without permission

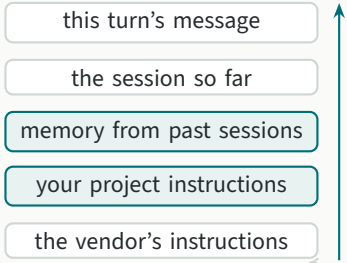
What Crosses the Wire



© Zhiyang Feng Redistribution prohibited without permission

- Nothing of yours moves and the model never logs in. What crosses is text: a tool call out, an observation back.
- Any file it reads crosses too. What it may open is an authorisation decision.
- **The meter counts both directions, and the part that grows is what your tools printed.**

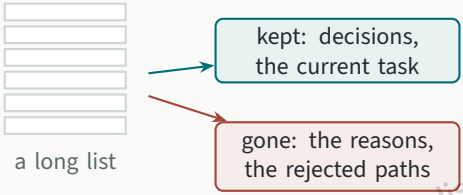
Five Layers of the Context Window



- Every turn the model decides from one place only: whatever is in this window now.
- The vendor's layer and this turn's message are not yours to manage. Your project instructions and memory are, and both are files on disk.
- **The binding constraint is attention, not size.**

© Zhaoyue Feng
Redistribution prohibited without permission

Compaction: What It Keeps, and What It Drops



- When the window fills, the harness summarises and carries on, on its own unless you do it first.
- Lossy in a specific way: conclusions survive; the reasons, and the small corrections you made in passing, often do not.
- **State belongs in files. Write down what must outlive the window, and compaction costs you nothing.**

© Zhigang Feng
Redistribution prohibited without permission

Tool, Skill, Sub-agent: Where the Reasoning Lives

tool	**skill**	**sub-agent**
code you wrote	prose you wrote	a second loop
the model picks the *arguments*	the model picks *to read it*	its own context and tools
no reasoning inside	reasoning stays in the main loop	reasoning happens elsewhere
costs a schema	costs one description line	costs a second context
fails by raising an error	fails by being ignored	fails by drifting off-task

© Lirang Feng
Redistribution prohibited without permission

- A tool is code you wrote: the model picks the arguments. A skill is prose you wrote: the model picks to read it. A sub-agent is a second loop with its own list.
- Loading order is cost order: rules at every start, a skill's body on match, a sub-agent in a fresh list.
- **Ask of any new agent term: once installed, where does the reasoning happen?**

A Skill Is a Folder: One Key File, the Rest Read Only When Needed

the folder

```
.claude/skills/                  in the project
└── fomc-library/                the name you type
    ├── SKILL.md                 the key file
    ├── scripts/toy_rag.py       run, not read
    └── references/cite.md       read if needed
```

when each part is read

- ☐ always in the list: name and description
- ☐ once used: the rest of SKILL.md
- ☐ on demand: a reference read, a script run

- **The folder holds a team's working knowledge, not a task.**

SKILL.md

when

```
---
name: fomc-library
description: Answers questions about FOMC
  statements from the local index, citing
  each claim as [file#n]. Use when asked
  what the Committee said at a meeting.
---
```

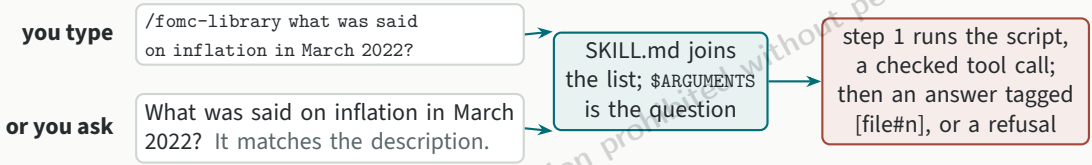
how

```
1. Run scripts/toy_rag.py on the question:
   ask index/ "$ARGUMENTS"
   (add --where fomc_YYYYMM for a meeting)
2. Answer only from the passages it prints;
   tag every claim with its [file#n].
3. If none answers, say so. How to cite:
   references/cite.md
```

© Zagros Feng Redistribution prohibited without permission

Source: Claude Code 2.1.303 skills documentation, read 21 October 2026; the open format: agentskills.io.

Two Ways to Start a Skill: Type Its Name, or Let the Model Match It



in the front block	you start it	the model	for
nothing (the default)	yes	yes	most skills, like this one
`disable-model-invocation: true`	yes	no	side effects: a deploy, a send
`user-invocable: false`	no	yes	background knowledge

- Unused, a skill costs its description; once used, its body stays in the list.
- **Write the description for the model: it decides from those lines alone.**

Source: Claude Code 2.1.303: skills documentation, read 21 October 2026.

© Zhigang Feng Redistribution prohibited without permission

Cheat Sheet: Commands, Keys and Modes

set up	`/init` write the brief · `/memory` edit it · `/permissions` allow, ask, deny · `/hooks` · `/skills` · `/agents` · `/mcp`
drive	`/plan` propose, change nothing · `/model` · `/effort` low to max · `/context` the window as a grid · `/compact` · `/clear` · `/resume`
check	`/diff` what changed · `/rewind` files, conversation or both · `/usage` cost and limits (`/cost`) · `/btw` a side question · `/export` · `/doctor` · `/help`
keys	Shift+Tab cycle the modes · Esc interrupt · Esc Esc rewind · Ctrl+O the transcript · Ctrl+B background a task · Ctrl+R history · Ctrl+C / Ctrl+D
prefixes	`/` a command or skill · `!` a shell command, its output into the session · `@` a file
modes	default (manual): reads only · acceptEdits · plan · auto: a second model, the classifier, reviews · dontAsk · bypassPermissions: isolated containers only

© Zhigang Feng Reproduction prohibited without permission

Four dials: what it reads, how hard it thinks, how much it may do unasked, what it costs.

Source: Claude Code 2.1.303: command help and documentation, read 18 October 2026.

Cheat Sheet: Files, Flags and Recovery

files it reads	`CLAUDE.md` the project brief (`AGENTS.md` is read in its place) · `~/.claude/CLAUDE.md` yours, every project · `CLAUDE.local.md` private, gitignored · `.claude/rules/*.md` · `.claude/settings.json`, `settings.local.json` permissions and hooks · `.claude/skills/<name>/SKILL.md` · `.claude/agents/<name>.md` · auto memory under `~/.claude/projects/`
flags on a cluster	`claude -c` continue · `-r <id>` resume · `--model` · `--effort` · `--permission-mode` · `--add-dir` · `--allowedTools` · `-p "<prompt>"` with `--max-budget-usd` · `--bg`, then `claude agents | attach | logs | stop` · `-w` a worktree · `--cloud`
recovery	connection cut → `tmux`, then `claude -r` in the same folder · window full → `/compact`, or `/clear` and a plan file · wrong turn → `/rewind` · broken setup → `claude doctor` · stale → `claude update`
habit	commit before you hand over the keyboard; read `git diff` before you believe a report

© Zhiqiang Feng
Redistribution prohibited without permission

State belongs in files; the session is a list you can resume, rewind or clear.

Source: Claude Code 2.1.303: command-line help and documentation, read 18 October 2026.

Two Dangers, Named Now

(i) the bottleneck shifts
everything the agent produces needs your judgement to verify; the queue moves to your desk

→

(ii) the black box
the production process behind a result is hard to disentangle, for you and for a referee

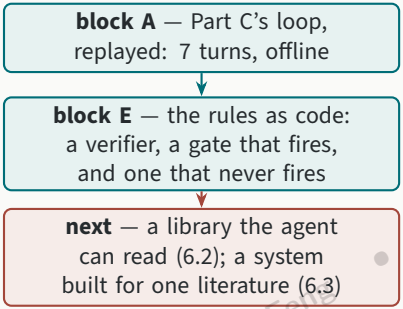
the evidence for (i) is Deck 1.1's survey of 637 scientists: the time the agent saved went to checking

- Both are real on the account in Part A; neither is an argument for going back.
- **The job is to make the advance work for us, and to name what it costs.**

Source: Codreanu, Imas, Mateos-Garcia et al. (2026), *AI in Science: Early Insights*, as read in Deck 1.1.

© Zhengyang Jiang
Redistribution prohibited without permission

Your Turn: Lab Blocks A and E — Then the Library, Then the Basis



- Self-paced, after class, offline: a CPU, no account, a clean kernel. About forty minutes for the two blocks.
- Block A's model is a script, not a network call, so your transcript and the recorded one agree line for line.
- **Next: a library the agent can read (6.2), and a system built for one literature (6.3).**

© Zhigang Feng Redistribution prohibited without permission

Source: The session's lab, blocks A and E, and their recorded-results files.

Session 6.2

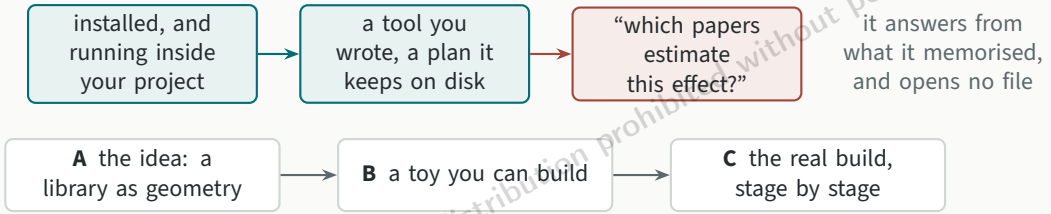
Build Your Own RAG: Give Your Agent a Library



Zhigang Feng

© Zhigang Feng
Redistribution prohibited without permission

Your Agent Runs Your Code, and Cannot Read Your Papers



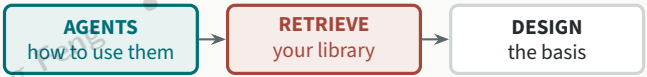
© Tingyu Fan Redistribution prohibited without permission

- An agent now runs commands, calls tools and keeps a plan on disk. The papers on your own disk are still outside the loop.
- Ask it which papers estimate your effect: it answers from what it memorised, and opens no file.
- **This deck puts your library inside the loop: the idea, a toy, then the real build.**

Session 6.2

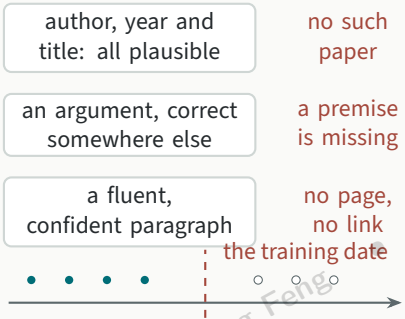
The Idea

Why the model cannot just answer, and the one move that fixes it



© Zhigang Feng
Redistribution prohibited without permission

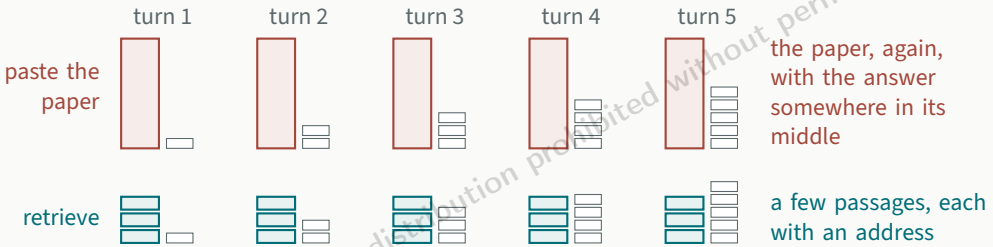
Three Symptoms, One Cause



- A fabricated citation, a method applied where it does not hold, a fluent paragraph with no page: three faces of one problem.
- The model answers from parameters frozen at a training date. Your papers were never in them, and nothing it says points to a page.
- **It does not have your literature, and what it has it cannot cite.**

© Zhaoyi Feng
Redistribution prohibited without permission

Pasting the Paper Is Not the Fix

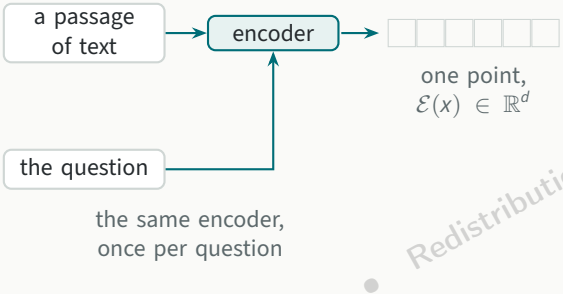


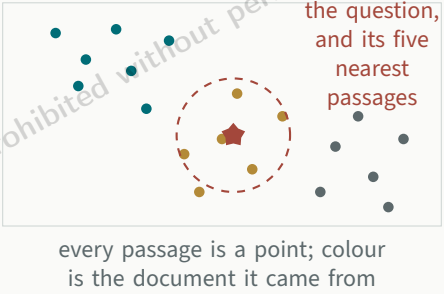
© Shuguang Sun · Redistribution prohibited without permission

- Paste a paper and every later question re-sends it; the passage you need sits in the middle, where a long context is used least reliably.
- Retrieval sends only the few passages the question needs, each with its address.
- **Send the evidence, not the library.**

Source: Liu et al. (2024), "Lost in the Middle", *TACL* 12, 157–173.

The Key Idea: Put the Library in a Space Where Nearness Means Meaning

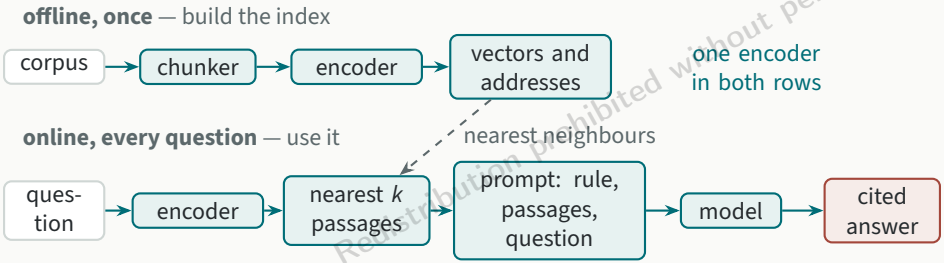




Redistribution prohibited without permission
© Zhiwei Feng

- Every passage is turned into a point by an encoder, once. Passages that say similar things land near each other: the geometry of word vectors, now for whole passages.
- A question is turned into a point by the same encoder. Its evidence is the nearest points, read back as text with their addresses.
- **The library becomes geometry, and search becomes nearest neighbours.**

Build Once, Ask Many Times

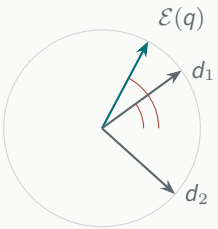


© Liyang Feng Redistribution prohibited without permission

- Offline, once: cut, encode, store the vectors beside their addresses.
- Online, per question: encode it, take the nearest passages, give the model one rule: answer from these, cite each.
- **Both rows use one encoder: similarity exists only inside one space.**

Source: Lewis et al. (2020), *NeurIPS* 33, 9459–9474.

Why Cosine: Compare Directions, Not Lengths



cosine compares angles, so passage length drops out

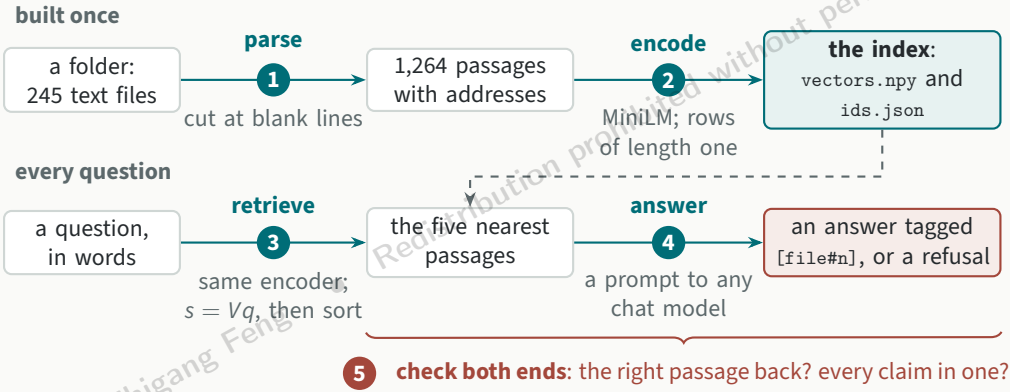
- Score a passage by the angle between its vector and the question's:

$$\mathrm{sim}(q, d_i) = \frac{\mathcal{E}(q)^\top \mathcal{E}(d_i)}{\|\mathcal{E}(q)\| \, \|\mathcal{E}(d_i)\|},$$

  and with unit-length vectors it is one dot product.
- **Top-k**: keep the k nearest. A **vector store** is the table of vectors and addresses that answers this quickly.
- **Nothing here understands the question; it measures angles.**

© Zhigang Feng
Redistribution prohibited without permission

A Toy You Can Build: Five Steps From a Folder to a Cited Answer



© Zhigang Feng Redistribution prohibited without permission

- Steps 1–2 run once, Steps 3–4 per question; Step 5's questions come first.
- **Every box is a thing you can open; each numbered arrow is one short listing.**

What the Toy Needs, Built Once: Parsing, Encoding, Storing

step	what you need	in the toy	it breaks when
1 parse	plain text saved as UTF-8; a rule that cuts it; an address on every piece	cut at blank lines; pieces over 40 characters kept; median 366 characters	the files are PDFs or scans: convert first
2 encode	`sentence-transformers`; one encoder, fetched once, then read from disk	all-MiniLM-L6-v2, a small BERT (6 layers), 91 MB; 384 numbers of length one; reads 256 word pieces, so 11 passages are cut	the question is read by another encoder
store	NumPy; the matrix and its addresses, in one order	`vectors.npy`: 1,264 × 384 numbers × 4 bytes = 1.9 MB; `ids.json`	one file is rebuilt, the other is not

- One install: `pip install sentence-transformers`; NumPy and PyTorch are in the course environment.
- **No server, no database: a laptop builds the index once, as two files.**

Source: The toy's run on 20 October 2026; the encoder's numbers from its own files.

© Zhiqiang Feng — Redistribution prohibited without permission

What the Toy Needs, Per Question: Retrieval, a Prompt, a Check

step	what you need	in the toy	it breaks when
3 retrieve	the same encoder; one product, one sort; a filter on the address for dates and names	$s = Vq$: 1,264 cosines from one product; the top $k = 5$ kept	the question names a date: right topic, wrong year
4 answer	a prompt (the rule, each passage with its address, the question) and any chat model	2,518 characters for the toy's question; the lab prints it and sends nothing	the rule or the addresses are left out
5 check	questions with known answers, and some the corpus cannot answer, written first	the lab's 26: 20 answerable, 6 not; the right passage back? each claim in one?	only the end that looks good is tested

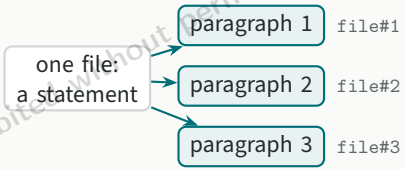
© Shuang Feng · Redistribution prohibited without permission

- Step 4 is the only step that needs a chat model: an API, a local model, or the agent itself.
- **Everything before the answer runs offline and gives the same rows on every run.**

Source: The toy's run on 20 October 2026; the lab's graded questions.

Step 1: A Folder of Text Becomes a List of Passages

```
from pathlib import Path
import json, numpy as np
docs, ids = [], []   # passage, address
for f in sorted(Path("corpus").glob("*.txt")):
    paras = f.read_text("utf-8").split("\n\n")
    for n, p in enumerate(paras):
        if len(p) > 40 and p[0] != "#":
            docs.append(p)
            ids.append(f"{f.stem}#{n}")
```



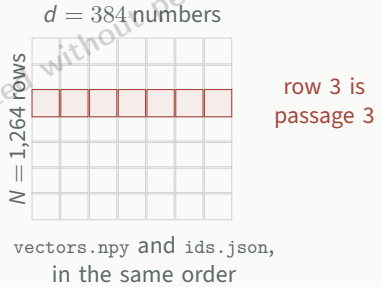
245 statements,
1994-02-04 to 2026-06-17,
become 1,264 passages

© Zhaoyang Feng · Redistribution prohibited without permission

- The corpus is a folder. Each file is read as text and cut at paragraph breaks into passages of a few hundred characters.
- Each passage keeps its address, file and position. That address is what a citation will print.
- **The unit you cut is the unit you will cite.**

Step 2: Encode Every Passage Once, Keep the Matrix

```
import sentence_transformers as st
name = "all-MiniLM-L6-v2"
enc = st.SentenceTransformer(name)
V = enc.encode(docs, normalize_embeddings=True)
# V: N x 384, one row per passage, length one
np.save("vectors.npy", V)
json.dump(ids, open("ids.json", "w"))
```

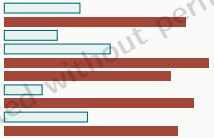


Redistribution prohibited without permission
© Thiguet Ferrer

- One call turns N passages into an $N \times d$ matrix, one row per passage; rows of length one, so cosine is a dot product.
- Save the matrix and the address list side by side. That pair is the whole index.
- **This is the expensive step, and it runs once.**

Step 3: Encode the Question, Take the Nearest Rows

```
q = enc.encode([question],
               normalize_embeddings=True)[0]
s = V @ q                  # cosine, all rows
top = np.argsort(-s)[:5]  # the five nearest
for i in top:
    print(ids[i], s[i].round(2), docs[i][:60])
```



s: one score per row; red, the five kept

N scores from one product, sorted once

printed, for *question* = “What did the Committee say about inflation in March 2022?”

`fomc_20090128#3`	0.66	In light of the declines in the prices of energy and other c…
`fomc_20180131#2`	0.66	Consistent with its statutory mandate, the Committee seeks t…
`fomc_20131218#2`	0.65	Consistent with its statutory mandate, the Committee seeks t…

- Right topic, wrong year: the encoder reads meaning, and the date lives in the address.
- **Search is one line of algebra; the date needs a filter on the address.**

Source: The listing on this page, run on the session’s 245 statements on 20 October 2026; the three best of its five rows.

© Zhiyang Geng 2026 Redistribution prohibited without permission

Step 4: Build the Prompt; Answer With Citations, or Decline

```
context = "\n\n".join(f"[{ids[i]}] {docs[i]}"
                      for i in top)
prompt = f"""Answer only from these passages.
Tag every claim with its [address].
If the answer is not in them, say so.

{context}

Question: {question}"""
answer = model(prompt)    # any chat model
```

the rule: only from these; tag each claim; if absent, say so

the passages: the five nearest, with addresses

the question

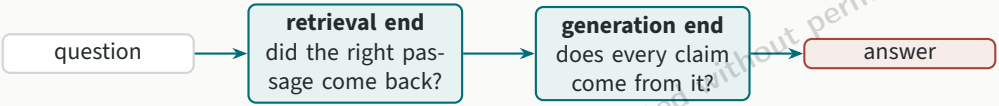
the answer: clauses tagged `[file#n]`, or "not in the passages"

Redistribution prohibited without permission

© Zhaney Feng

- Three parts: the rule, the passages with their addresses, the question. Given the rows of Step 3, the right answer is a refusal: the model fills no gap from memory.
- The only place a generative model runs. Everything before it exists to put good evidence in front of it.
- **"Not in the passages" is the system working; a fabrication costs the paper.**

Step 5: Check Both Ends Before You Trust It



a good score at one end and a bad one at the other is not a good system: it is a system that fails in a way you have not measured

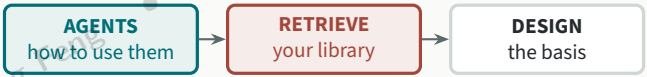
- Before you build, write questions with known answers, and some the corpus cannot answer.
- Retrieval end: did the right passage come back, recall at k? Generation end: does every claim in the answer appear in the passages, **faithfulness**? A refusal rule is a threshold on the best score.
- **Both ends must pass. They fail for different reasons and are repaired in different places.**

© Aakash Kumar Nain · Non-distribution prohibited without permission

Session 6.2

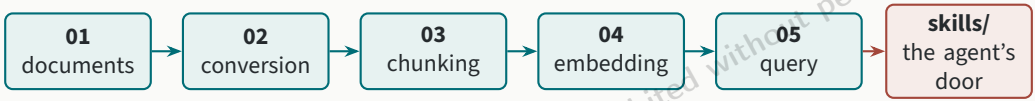
The Real Build, Stage by Stage

The toy’s five steps, each now a folder, a tool and a check



© Zhigang Feng · Redistribution prohibited without permission

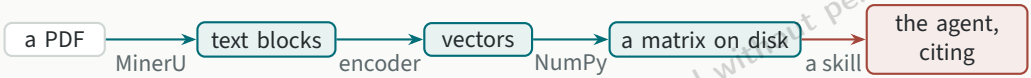
The Build Is Five Numbered Folders



© Zhigang Feng Redistribution prohibited without permission

- A stage reads the folder before it, writes its own; a failure is fixed in one place.
- The toy's five steps, now five folders: what changes is scale, formats, checks.
- **Number the stages, and never let one write into another.**

The Four Tools, in Plain Words, and Where You Met Each

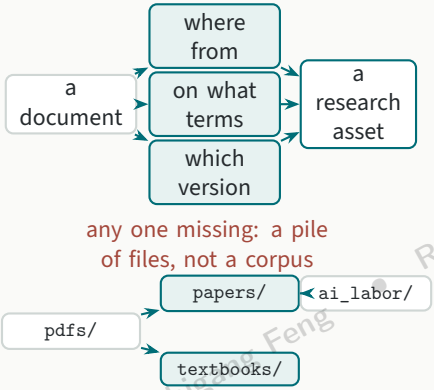


the tool	how it works	you met it as
MinerU, a layout model	reads a page as a picture and writes out its blocks, each with its type and page	5.1: a PDF is a picture, not tokens; 5.3: read, write
the encoder, BGE-M3, open weights	reads a passage both ways into one unit vector; two languages, one space	5.2: word vectors, cosine; 5.3: the pinned version
a NumPy store, the matrix on disk	one row per chunk; search is one product Vq and a sort, a query against its keys	Sessions 2–4: matrix products; 5.2: query–key scores
a skill: a manual and a script	read only when a question needs the library: when to run the script, how to cite	6.1: a folder opened when needed; the loop with a tool

- Retrieval is attention over a library: the question is the query, the vectors are its keys.
- **No magic: a reader for pictures, an encoder for meaning, a matrix, a manual.**

© Zhigang Feng. Reproduction prohibited without permission

Stage 1, Documents: The Corpus Is a Choice, and the Folder Is Its Metadata

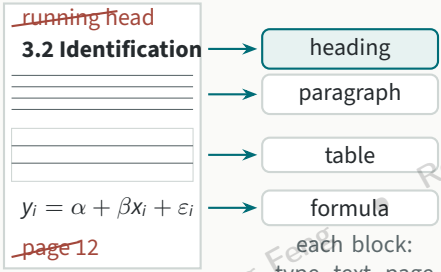


- What is left out cannot be retrieved. No licence, no provenance, no frozen version: a pile of files, not a corpus a referee can re-run.
- The path is the topic, and a manifest row carries each file's content hash, so a rerun passes over what is already converted.
- **The cheapest metadata is the kind you cannot forget to update.**

© Zhaoran Feng · Redistribution prohibited without permission

Stage 2, Conversion: A PDF Becomes Blocks of Text, Each With Its Page

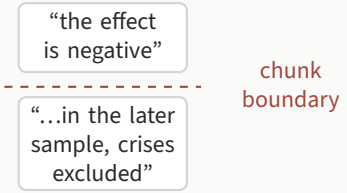
```
mineru -p staged/ -o markdown/ -b vlm-auto-engine  # one engine start per batch
pandoc paper.tex -t gfm -o paper.md                # TeX, HTML, EPUB: no model
```



- A PDF is a picture. A layout model on a GPU reads it into blocks, heading, paragraph, table, formula, each with its page; running heads and page numbers are dropped.
- The engine loads once, then every page is cheap: stage the batch, start it once. Other formats need no model.
- **Normalise early: every format becomes one block record, and the page travels with the text.**

© Zhigang Feng Redistribution prohibited without permission

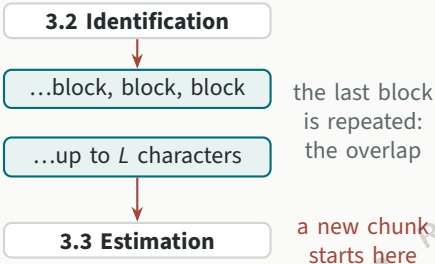
Stage 3, Chunking: Why Cut, and the Trap



- A **chunk** is the unit retrieval returns and the unit a citation names. Too small, it loses its context; too large, it mixes subjects and crowds the prompt.
- The trap is a claim cut from its condition: the sample, the specification, the date. Retrieved alone it reads as general, and is not.
- **Cut at the unit you would be willing to cite.**

© Zhigang Feng
Redistribution prohibited without permission

Stage 3, the Rule: Headings Are Hard Boundaries, and What One Chunk Carries



- Blocks under one heading accumulate to a target length L with a floor ℓ; the last block repeats into the next chunk as overlap, never a cut sentence, never across a heading.
- The working system: $L = 1200$ and $\ell = 200$ characters, overlap by whole blocks up to 150, read from its settings file.
- Each chunk carries its address: document, page range or section path, and its heading.
- **A chunk that spans two sections answers neither question.**

© Zhigang Feng
Redistribution prohibited without permission

Two Encoders, One Recipe: MiniLM for the Toy, BGE-M3 for the Library

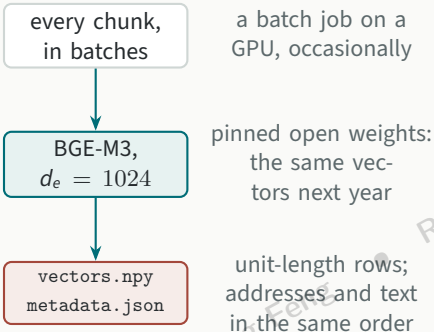
	the toy: all-MiniLM-L6-v2	**the library**: BGE-M3
built on	MiniLM: a 6-layer BERT, English	XLM-RoBERTa: 24 layers, 100+ languages
a passage becomes	384 numbers: its tokens averaged	1,024 numbers: its first token's vector
reads at most	256 tokens (11 of 1,264 passages cut)	8,192 tokens
trained on	over a billion pairs	pairs, after long-text pre-training
size; runs on	22.7 M parameters, 91 MB; a laptop	568 M parameters, 2.3 GB; GPU to build

- BGE-M3 has two more outputs, a weight per word (like a keyword score) and a vector per token. The build keeps only the one point.
- Change the toy's year: MiniLM keeps cosine 0.89, BGE-M3 0.80. Change its topic: 0.63 and 0.83. Neither replaces a filter on the address.
- **Vectors on disk are comparable only with the weights that wrote them: pin the file, not the name.**

Source: Model cards and configuration files of all-MiniLM-L6-v2 and BAAI/bge-m3, read 20 October 2026; Chen et al. (2024).

© Zhicheng Feng · Redistribution prohibited without permission

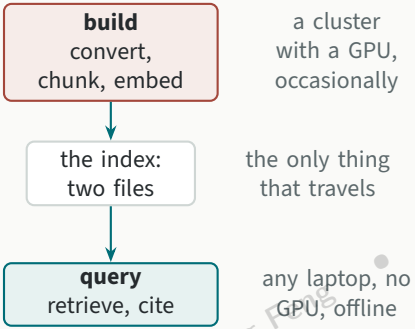
Stage 4, Embedding: One Open-Weight Multilingual Encoder, Vectors on Disk



- Choose the encoder on four things: languages, domain fit, passage length, and whether the weights are yours to pin. One model encodes corpus and question alike.
- A batch job on a GPU writes two files: unit-length vectors, and addresses and text in the same order. The build refuses vectors from any other model.
- **Open weights are a replication requirement: a hosted encoder can change under you; a pinned one re-runs.**

© Zhigang Feng Redistribution prohibited without permission

Build on a GPU, Query Anywhere



- The expensive half runs rarely, where the GPU is. The half you use daily needs only the index and the encoder, on any laptop, with no network.
- Two environments on purpose: the query side never installs the GPU build of the deep-learning library. NumPy carries the store until the corpus passes a few hundred thousand chunks.
- **This split is the one design decision that makes a research RAG system livable.**

© Zhigang Feng Redistribution prohibited without permission

Stage 5, Query: Encode, Score, Rank, Hand Over

```
[doc-07 | title | p. 12 | 0.71]
§ 3.2 Identification
the passage, as converted, its heading
above it
[doc-12 | title | pp. 4--5 | 0.66]
…
[doc-03 | title | §2.1 | 0.64]
… k entries, best first
```

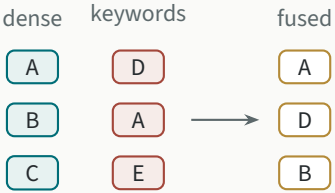
Answer only from the above. Cite each claim as [title, page]. If the evidence is weak, say so.

printed: the evidence, then the rule

- The question is encoded by the same model and scored against every vector in one product; the top k come back with their addresses. A topic filter applies *after* ranking, so a narrow literature never starves the result.
- The script prints evidence and an instruction. It never calls a language model itself.
- **Everything upstream exists to put good evidence in front of the model.**

© Zhigang Feng
Redistribution prohibited without permission

Two Upgrades When Dense Search Misses: Keywords, and a Second Reader



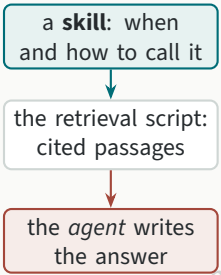
A tops one list and is second in the other, so it wins the merge

- Dense search can walk past an exact term; a keyword score (**BM25**) counts it. Merge the two lists by rank: $\mathrm{RRF}(d) = \sum_r 1/\big(\kappa + r(d)\big)$, $\kappa = 60$, a default fixed in a pilot. The lab's block B does this.
- A **re-ranker** reads question and chunk together, for the shortlist only; what the first pass missed it never sees.
- **Add these when a graded question set says the first pass misses, not before.**

Source: Cormack, Clarke & Büttcher, *Proceedings of SIGIR '09* (2009), 758–759.

© Zhigang Feng
Redistribution prohibited without permission

Make It a Tool the Agent Can Reach: A Skill



a second skill at ingest: read the front page, fill in author and year

- A skill folder tells the agent when and how to run the retrieval script; it reads it when a question needs the library.
- The agent writes the answer from the printed evidence: every claim tagged with title and page, or “insufficient evidence in the corpus”.
- **The generator is the agent you already have. The library is one more tool.**

© Zhigang Feng
Redistribution prohibited without permission

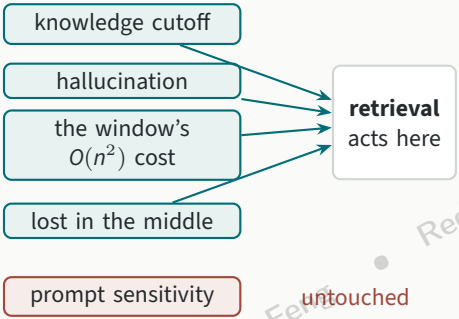
Where It Breaks, and Which Stage to Repair

What you see	Where it sits	What repairs it
nothing relevant comes back	the index	the corpus boundary
the right passage ranks fourth	the ranking	a second scorer, or re-ranking
a claim lost the condition it held under	the chunker	boundaries that respect structure
a confident answer about a year you never indexed	the query	a refusal rule
a confident answer from the wrong meeting	an ambiguous query	ask it back before answering

© Zhigang Feng
Redistribution prohibited without permission

The lab produces three on purpose: a stale index, a cut inside the decision sentence, an ambiguous question answered from the wrong meeting. **The symptoms read alike from outside. Diagnose by asking which stage could have produced this one.**

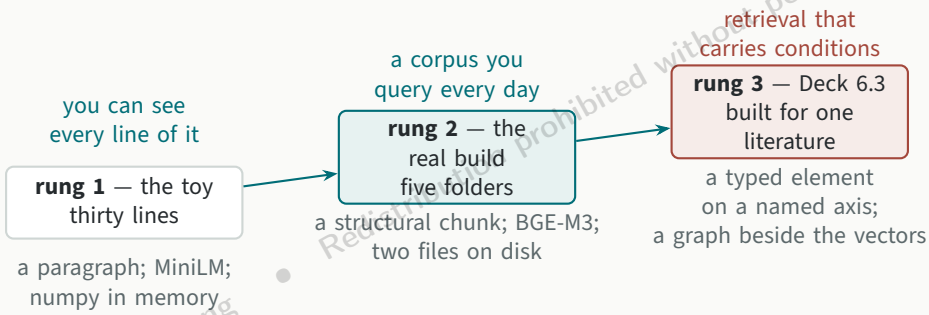
The Five Limits of Session 5, Four Repaired



- Session 5 closed on five limits of a pretrained model, all architectural: a larger model does not remove them.
- Retrieval repairs four by one move: the knowledge left the parameters for an index you can rebuild today. Prompt sensitivity is untouched; it belongs to the prompt, not the evidence.
- **Two remain: the refusal is a rule you write, and a dense vector cannot tell you when it is wrong.**

© Zhigang Feng Redistribution prohibited without permission

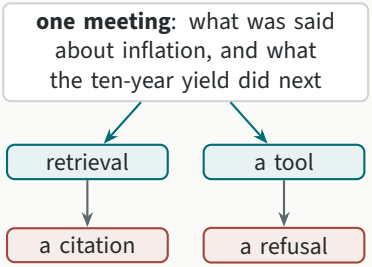
Three Rungs: Toy, Working, Research



© Zhengyang Feng
Redistribution prohibited without permission

- Each rung answers what the rung below answers badly, and costs more to keep standing.
- **Climb when the rung below is boring.**

Your Turn: Lab Blocks B, C and D



drop any one and the task is still answerable — fluently, and wrongly

- **B**, build the index on the 245 statements: chunks that name their statement, two lists fused by rank. **C**, answer or refuse by a fitted threshold, with three failures made on purpose. **D**, mark one answer's claims by hand.
- B opens by building this deck's toy, then hands you its five steps as one file for your own folder.
- **Self-paced, after class, on a CPU; the encoder is fetched once.**

Source: The session's lab, blocks B to D; Session 2's frozen yield panel.

© Zhigang Feng Redistribution not allowed without permission

Session 6.3

GRAM: Generative Reasoning and Augmented Modeling for Quantitative Macroeconomics



Redistribution prohibited without permission

© Zhigang Feng

Zhigang Feng

This deck describes a system that is work in progress by Zhigang Feng, Jianjun Miao, Haoxuan Xie, Yaolong Zhong, and Yingli Zhou (2026). Its evaluation is a design; no evaluation result is reported here.

Session 6.3

What We Do

What this paper is about, four ways to give a model context, a named basis, and the pilot



© Zhigang Feng · Redistribution prohibited without permission

What This Paper Is About

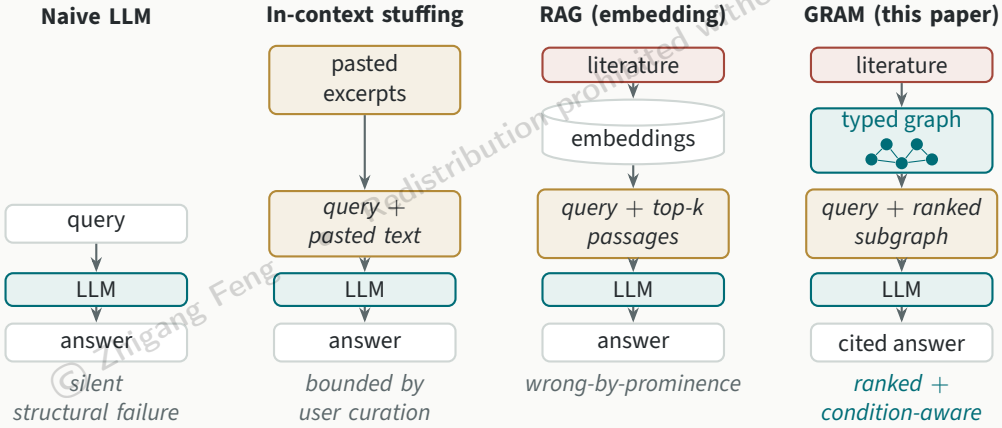
We build **GRAM**: it grounds an LLM's reasoning, answers, and code in the field's **own literature**, built **once** into a searchable knowledge base.

- **The core idea — PromptEcon**: read each paper against a **fixed basis**, a standard for describing any structural macro model. Every paper lands on the **same coordinates** and links to the rest; with a citation graph, this forms one searchable knowledge base. *(Built in 5 stages: parse → citation graph → model extraction → semantic citations → embed.)*
- **At query time the context helps two ways**: *(a)* steer what the model *already* knows; *(b)* add what it does *not* know (post-cutoff papers, exact identities). Because the basis is typed, retrieval is **condition-aware** — a method comes with the conditions under which it holds.
- **Pilot on OLG, but not specific to it**: a reproducible recipe for turning *any* structured economics literature into a knowledge base for AI-augmented research.

© Zhiheng Feng. Redistribution prohibited without permission

Same Query, Four Context Mechanisms

"How do we solve for the recursive competitive equilibrium of this OLG model?"



© Zingang Feng — Unauthorized distribution prohibited without permission

The Four Mechanisms, Side by Side

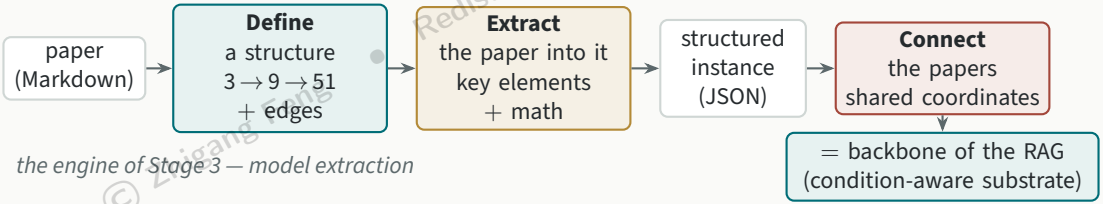
Property	Naive LLM	In-context stuffing	RAG (embedding)	GRAM (this paper)
Fact lookup	Partial	Partial	Yes	Yes
Compositional retrieval	No	Partial	Partial	Yes
Dependency-aware retrieval	No	User-supplied	No	Yes
Condition-aware retrieval	**No**	**No**	**No**	**Yes**
Scales to large corpora	—	No	Yes	Yes
Algorithm transfer	Weak	Moderate	Weak	Strong

© Zhigang Feng
Redistribution prohibited without permission

PromptEcon: What It Is, and Its Role

PromptEcon is the engine of the extraction stage (**Stage 3**). Reading each paper with an LLM, it does three things:

- **Defines a structure** — a fixed basis (**3 blocks** → **9 categories** → **51 axes** + 7 typed edges): the same coordinates for every paper.
- **Extracts each paper into it** — key elements (prose + inline math) become one structured instance.
- **Connects the papers** — shared coordinates let papers align by typed concept: the **backbone of the RAG**.



the engine of Stage 3 — model extraction

© Zhigang Feng
Redistribution prohibited without permission

*This basis is what makes retrieval **condition-aware** (the bold row earlier). Built once, in Stage 3.*

Why 51 Axes? Projecting a Paper onto a Fixed Basis

	an asset	a paper
named basis	Fama–French factors: market, size, value	the 51 axes: `preferences`, `technology`, …
statistical basis	PCA: components 1, 2, 3, …	the 4096-d embedding: $v_1, \ldots, v_{4096}$

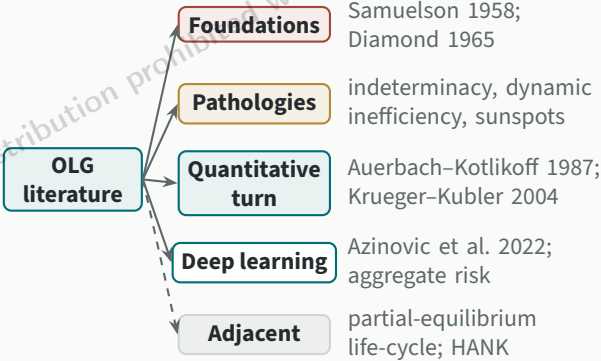
- The 51 axes are a **fixed basis**: extraction projects each paper onto the *same* named coordinates, so a whole paper becomes a short **coordinate vector**.
- An **empty axis** is a zero coordinate — still information: the model has no such ingredient.
- **Named, like Fama–French's factors: chosen by theory, so every coordinate is interpretable and auditable, and any two papers compare axis by axis.**

© Anyang Feng · Redistribution prohibited without permission

Our Pilot: Overlapping-Generations (OLG) Models

Why OLG?

- **Active frontier:** large OLG models with aggregate risk are still an open computational problem.
- **Crisp structure:** each ingredient is forced by a pathology that representative-agent models lack.
- **Author expertise:** we can validate every extraction; the corpus is curated, not scraped.



© Zhigang Feng · Redistribution prohibited without permission

What the Infrastructure Does: GRAM in Two Modes

The same knowledge base supports two modes of help — the two halves of GRAM:

Generative Reasoning
The LLM reasons over a *retrieved typed subgraph* to answer field and research questions — grounded and cited.

Augmented Modeling
The same knowledge base scaffolds *designing algorithms, writing, and validating* model code.

- Not a generic chatbot that paraphrases textbooks — an assistant that **respects the structure** of the field.
- Every paper is **math** (analytical scaffold) *and* **prose** (model anatomy, mechanisms); the assistant reads both.
- Operationalised by a **structured graph** over a **curated literature** — not a flat embedding store.
- **Pilot:** OLG in quant macro; the recipe generalises to other fields of economics and social science.

Nothing like this exists for economics today.

© Zhiyang Feng Redistribution prohibited without permission

Where This Sits

Two research lines retrieve over documents that are mostly *math* or mostly *prose*; economics papers are both.

Math information retrieval	Domain social-science RAG	This paper (econ)
Pure-math papers (Ju et al. 2026, …)	Qualitative prose (Legal: Chen et al. 2026, ACL; Medical: variants)	*Both at once.*
Unit: theorem / lemma / equation.	Unit: case / statute / clinical note.	Unit: typed economic axes, equations carried inside.
Structure: emergent from embedding similarity.	Structure: typed graph nodes (case, statute, …) inject domain knowledge.	Structure: typed graph; domain knowledge from textbook canon.
Strong on math, blind to prose.	*Strong on prose, no math.*	*Math is analysis; prose is mechanism.*

One extraction carries both — each populated axis holds the prose anatomy with its math inside. No direct analogue in economics today.

© Zhigang Feng Redistribution prohibited without permission

Session 6.3

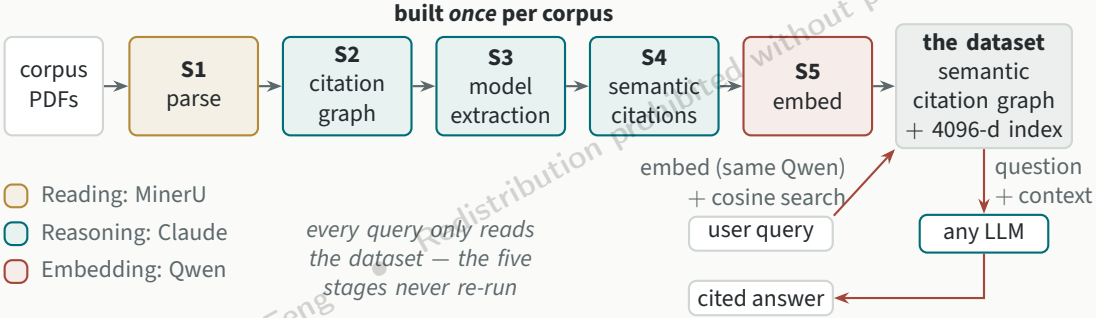
Infrastructure Development

A one-time build: three LLMs, a curated corpus, five stages, and a basis with names on its axes



© Zhigang Feng · Redistribution prohibited without permission

One-Time Build, Then Unlimited Queries



© Zhitang Feng · Redistribution prohibited without permission

The dataset is built **once** per corpus by the five-stage pipeline (S1–S5); thereafter every query only *reads* it — never re-running extraction.

Three LLMs do the building, each in a distinct role.

Three LLMs, Three Roles

The pipeline calls three different LLMs, each for what it is best at; they never substitute for one another.

Role	Model	Modality	Maps
Reading	MinerU	vision–language	page *images* → structured Markdown
Reasoning	Claude	generative (text→text)	text → structured knowledge / prose
Embedding	Qwen3-Embedding-8B	embedding (text→vector)	text → 4096-d vector

- **Reading** (S1): reconstructs each PDF page — OCR, layout, reading order, tables, equations → LaTeX.
- **Reasoning** (S2–S4): the judgment work — resolving citations (S2), **extracting each paper onto the 51 axes** (S3), and summarising how each paper uses what it cites (S4).
- **Embedding** (S5): maps every chunk — and, at query time, the question — into one shared 4096-d space.

Redistribution prohibited without permission
© Jingyang Yang

Curation: Tiered Selection of the OLG Corpus

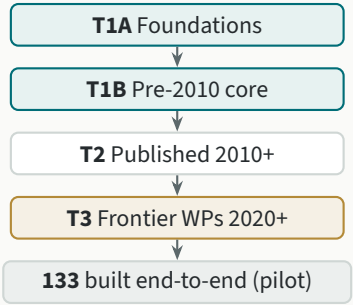
Three design pressures shape the list: **coverage vs. focus • pre- vs. post-cutoff • network connectivity**.

Tier 1A — Foundations. Pre-1980 load-bearing papers anchoring the theory.

Tier 1B — Pre-2010 core. Mechanism and computation pillars: pathologies, numerical methods, bequests, inequality.

Tier 2 — Published 2010–present. Top and field-journal papers spanning the full quant-macro stratum.

Tier 3 — Frontier WPs 2020+. NBER, SED / Summer Institute, arXiv, journal in-press, and job-market scans.



We treat corpus construction as engineering, not bibliography.

© Zheng Feng • Redistribution prohibited without permission

Stage 1: Document Parsing (the Reading LLM)

Economics PDFs are not plain text: two-column layouts, displayed equations, tables, and figures carry the model, and a naive text dump scrambles reading order and drops the math. So we *read* each page with a vision–language model. MinerU's `vlm-auto-engine` looks at the rendered *page image* and reconstructs it as machine-readable Markdown — OCR, layout and reading-order recovery, tables, and equation → LaTeX, in one pass. Output per paper:

- `<id>.md` — layout-aware Markdown, LaTeX math preserved **(used by Stages 3–4)**.
- `<id>_middle.json` — structured layout: pages → typed blocks, reading order, and the parsed reference list **(used by Stages 2–3)**.
- `_content_list_v2.json` / `_model.json` — intermediate VLM state, re-runnable; `_layout.pdf` for visual QA; `*.jpg` images for extracted figures.

© Zhigang Feng • Redistribution prohibited without permission

Stage 2: Citation Graph (the Reasoning LLM)

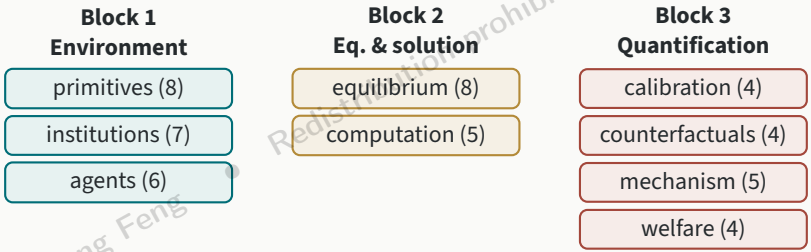
Who builds on whom, across the corpus. For every paper we take its reference list and ask: which of these references are themselves *in our corpus*? An LLM matches each messy reference string to a corpus paper (or to none), giving a clean "*X* cites *Y*" graph over the corpus:

1. One Claude subagent per citing paper — fresh context, parallelisable.
2. Each resolves every listed reference to an in-corpus paper id (or none): bibliographic **0/1 edges**, bare ids only.
3. The script assembles the responses into `citation_graph.json` — a directed adjacency over the N papers, i.e. an $N \times N$ matrix.

© Zhigang Feng • Redistribution prohibited without permission

Stage 3: A Standard Basis for the Field

Stage 3 runs **PromptEcon**: it reads each paper against a **fixed basis** — a standard for describing any structural macro model. Its $m = 51$ **axes** are named coordinates; extraction *projects the paper into this m-dimensional space*, populating only the axes it has:

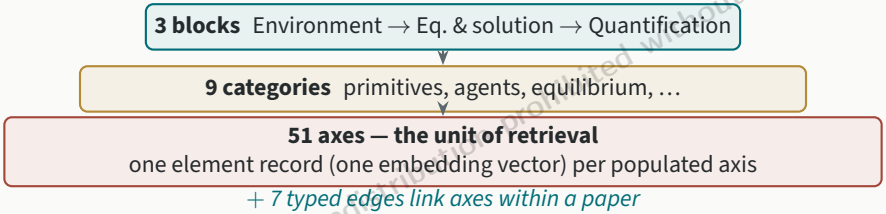


© Zhigang Feng
Redistribution prohibited without permission

9 categories group the 51 axes across 3 blocks. A paper fills each axis at most once — one short titled statement, with its equations written inline — and an axis left empty is itself informative (the model simply has no such ingredient).

The Framework: Its Building Blocks

Stage 3 reads each paper against this fixed structure:



© Zhiyang Ong
Reproduction prohibited without permission

Term	#	Definition
Block	3	Environment → Equilibrium & solution → Quantification.
Category	9	finer grouping inside the blocks (primitives, agents, …).
Axis	**51**	the **unit of retrieval**: one `element` record per populated axis (one embedding vector each).
Typed edge	7	within-paper dependency between axes (`defines`, `parameterizes`, `clears_in`, …).

The Basis, All 51 Axes: Block 1 (Environment, 21)

Category	Axes
primitives (8)	`time`, `demography`, `type_space`, `preferences`, `endowments`, `technology`, `exogenous_shock_process`, `state_space`
institutions (7)	`market_structure`, `friction`, `borrowing_constraint`, `government_rule`, `monetary_regime`, `contract`, `corporate_structure`
agents (6)	`household_problem`, `firm_problem`, `government_problem`, `intermediary_problem`, `planner_problem`, `information_structure`

*Block 1 captures **what the world contains** (primitives), **the rules of the game** (institutions), and **who optimises** (agents) — the Hurwicz–Myerson three-way split.*

© Zhigang Feng Redistribution prohibited without permission

The Basis, All 51 Axes: Blocks 2 and 3 (30)

Block	Category	Axes
Equilibrium & solution	equilibrium (8)	`state_vector`, `policy_function`, `price_function`, `optimality_condition`, `market_clearing`, `distribution_evolution`, `transversality_condition`, `equilibrium_concept`
Equilibrium & solution	computation (5)	`solution_method`, `discretization`, `distribution_approximation`, `accuracy_diagnostic`, `numerical_implementation`
Quantification	calibration (4)	`parameter`, `target_moment`, `estimation_procedure`, `data_source`
Quantification	counterfactuals (4)	`benchmark_economy`, `alternative_regime`, `comparison_metric`, `transition_path`
Quantification	mechanism (5)	`causal_channel`, `structural_argument`, `comparative_static`, `theoretical_result`, `intuition_narrative`
Quantification	welfare (4)	`welfare_metric`, `welfare_evaluation_point`, `welfare_decomposition`, `welfare_result`

© Zhigang Feng
Redistribution prohibited without permission

Basis Design: How Big, and Where It Comes From

- **How big?** $m = 51$ is a deliberate *dial*, not a limit — a larger basis gives finer resolution at higher curation cost; the precedent is ImageNet (Deng et al. 2009), whose finite, human-curated label inventory was enough to transform computer vision.
- **Where from?** The 9-category split is assembled from five canonical treatments — Arrow–Debreu GE, recursive macro (SLP), MDPs (Bertsekas), mechanism design (Hurwicz–Myerson), and the quantification canon — so every category and axis is auditable by reference.

© Zhigang Feng — Not to be distributed or published without permission

Per-Paper Instance: One JSON, Fully Provenanced

Each paper produces one `<id>.json`: a list of `element` records (one per populated axis) plus an `edges` array; each `element` becomes one embedding chunk in Stage 5. Example below from the OLG run (`abel_panageas_2023`, abridged).

```
{ "model_id": "abel_panageas_2023",
  "elements": [
    { "kind":           "market_clearing",
      "category":       "equilibrium",
      "canonical_name": "capital-market clearing (saving = capital + bonds)",
      "description":    "Young saving finances all assets, so capital is
                         saving net of bonds: $K_{t+1} = S_t - B_{t+1}$.",
      "source_lines":   [[612, 619]] } ],
  "edges": [
    { "from": "government_rule_001", "to": "market_clearing_001",
      "type": "enters_constraint" } ] }
```

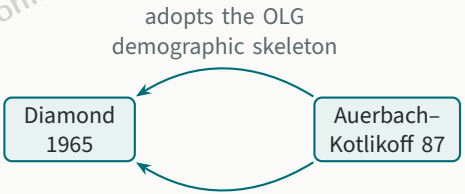
`kind` names the **axis** (1 of 51); `category` its group (1 of 9); `description` (prose + inline math) is the **only field embedded**; `source_lines` give provenance; `edges` link the paper's axes by one of the 7 types.

© Zhiyuan Feng · Redistribution prohibited without permission

Stage 4: Semantic Citations, How *X* Uses *Y*

Stage 2 gives only the bare "*X* cites *Y*" graph. Stage 4 fills in *how* each citation is used — adding no new edges.

- **What:** for each citation, a one- to two-sentence note — does *X* adopt *Y*'s model, borrow its method, calibrate to it, or benchmark against it?
- **How:** an LLM reads the sentences around the citation in *X* and writes that note; one pair can get several notes.
- **Why natural language** (not a fixed label): the note becomes a search vector *as-is*, so a citation is retrievable by meaning — a one-word label like "extends" cannot be, and hides *what* is extended.



multi-edges per pair, each with its own natural-language summary.

© Zhigang Feng · Redistribution prohibited without permission

Stage 5: Embedding (the Embedding LLM)

Goal: make the extracted pieces searchable. We turn each `element` record and each citation note into one vector — so the corpus becomes **machine-readable and indexed**; similar economics lands at nearby vectors, and a query then retrieves **by meaning, not keywords**. The unit we embed is the axis (one `element` record), not the whole paper or section.

- **Model:** Qwen3-Embedding-8B turns each chunk into one **4096-d** vector, length-normalised so a similarity score is just a cosine (a dot product).
- **What a chunk is:** one per `element` record and one per citation note — the same units we later retrieve.
- **Output:** a vector matrix plus aligned metadata (paper, axis, category, provenance); new papers are appended without re-embedding the rest.

*The **same model** embeds the query at retrieval time — so question and corpus share one space.*

© Zhigang Feng Redistribution prohibited without permission

What Each Stage Produces: From Papers to a Searchable Index

Input: N selected papers. Each stage adds structure:

Stage	Produces	Shape
S1	machine-readable documents	N Markdown / JSON files
S2	citation topology	$A \in \{0,1\}^{N \times N}$
S3	each paper projected onto the basis	up to m element records per paper ($m = 51$)
S4	usage text on every citation	refined $N \times N$ (edges carry summaries)
S5	everything embedded	each element & usage in $\mathbb{R}^{4096}$

© Zhigang Feng • Redistribution prohibited without permission

Where Each Stage Runs

Stage / job	Hardware	Software
S1 parse	T4 GPU	MinerU (vision–language model).
S2–S4 reason	— (API)	Claude.
S5 embed	T4 GPU	Qwen3-Embedding-8B.
Curation / glue	CPU	Python (`llm_econ`).

© Zhigang Feng Redistribution prohibited without permission

Session 6.3

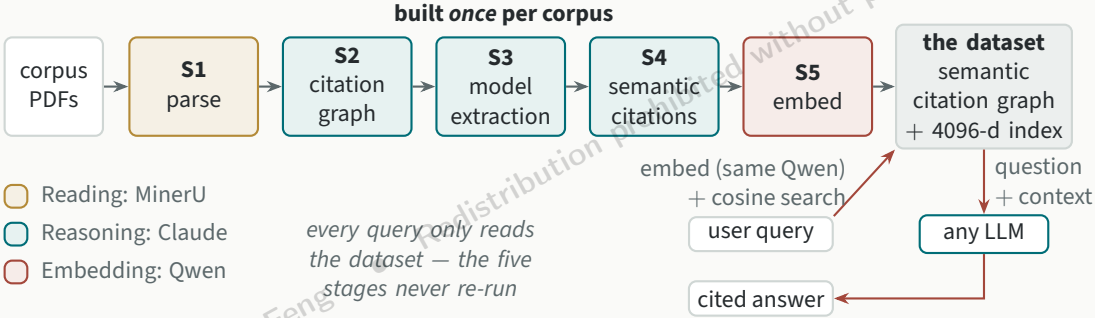
Generative Reasoning and Augmented Modeling

The read-only query loop, two demonstrations, and the recipe in other fields



© Zhigang Feng · Redistribution prohibited without permission

Recap: We Built It Once; Now We Query It



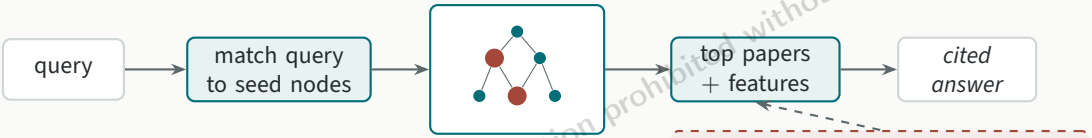
© Jiacheng Feng & Yutong Xie. Redistribution prohibited without permission

The five stages (S1–S5) ran **once** to produce the dataset — a semantic citation graph plus a 4096-d index. Everything that follows is the **read-only query loop** (bottom row): no stage ever re-runs.

Two ways to use it — ask the corpus directly, or ask any research question and let retrieval ground the answer.

At Query Time (1/3): The Intuition

What actually happens when you ask a question:

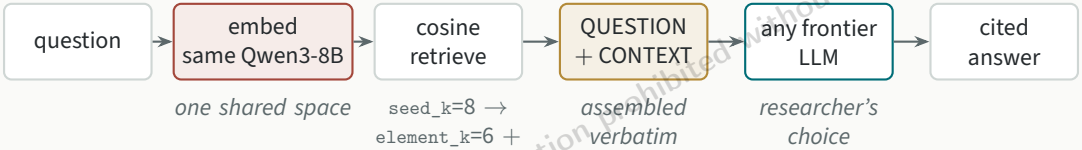


© Zhigang Feng Redistribution prohibited without permission

- The **same** embedder that built the index maps your question into the *one shared space* as the corpus.
- The nearest *typed pieces* — model elements and citation notes — are pulled **by meaning**, not keyword overlap.
- The model answers **only from those pieces**, with citations; off-topic or inadmissible material sits far away and is filtered out *before* the model reads anything.

At Query Time (2/3): The Mechanics

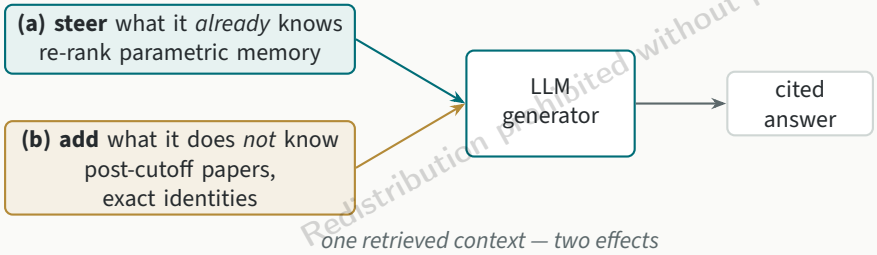
After the one-time build, every query is read-only and deterministic:



Redistribution prohibited without permission
© Zhaoyang Wang

1. The researcher types a question (`/promptecon "..."`) — a field question, an algorithm to design, eventually code to write.
2. The **same Qwen3-Embedding-8B** embeds it, so query and corpus live in **one semantic space**.
3. Cosine retrieval: the `seed_k`=8 nearest hits seed the search; the context keeps its top `element_k`=6 element + `citation_k`=4 citation chunks (10 in total).
4. A verbatim QUESTION + CONTEXT block goes to **whatever frontier LLM the researcher picks**; the answer is written *from that block alone* — cite-or-decline, with provenance.

At Query Time (3/3): Two Ways the Context Helps



Redistribution prohibited without permission

© Siqing Fang

(a) Steer what the model knows
Targeted context re-ranks the generator's *own* parametric knowledge toward the right region of the field.

(b) Add what it does not know
The same context injects post-cutoff papers, exact accounting identities, and quantitative results absent from its weights.

Demo 1: Corpus-Targeted Query (the Prompt)

A question that names the corpus directly:

Question. What computational or solution methods do these papers use to solve OLG models with aggregate risk?

Retrieval builds the literal augmented prompt (abridged):

```
CONTEXT (retrieved from the OLG GraphRAG):
[1] nishiyama_smetters_2013 -> hasanhodzic_kotlikoff_2013 (citation)
    state-of-the-art: simulation-projection on an 80-period OLG with aggregate risk.
[3] hasanhodzic_kotlikoff_2013 -> rios_rull_1996 (citation)
    55-period OLG with aggregate shocks solved by local perturbation.
[6] azinovic_yang_cole_kubler_2023 :: solution_method
    simulation-based Krusell-Smith / Kubler-Schmedders with endogenous grid method;
    Euler equations inverted along simulated aggregate paths, then damped iteration.
[10] krueger_mitman_perri_2016 :: solution_method
    Krusell-Smith bounded-rationality Euler solver; occasionally binding constraints.
...  (10 chunks: 6 element + 4 citation, across 9 papers)
```

rank=cosine, seed_k=8, element_k=6, citation_k=4.

(c) Thomas Fearnley Redistribution prohibited without permission

Demo 1: The LLM's Answer (Verbatim, Abridged)

Answer — written from the augmented prompt alone

The retrieved evidence shows these papers solve aggregate-risk OLG models with a handful of method families:

- **Krusell–Smith / simulation-based Euler** (the dominant approach): `azinovic_yang_cole_kubler_2023` solves the household / firm / market-clearing system *only along simulated paths*, with the **endogenous grid method** inverting the consumption Euler equations; `krueger_mitman_perri_2016` uses the same idea with occasionally-binding constraints.
- **Simulation–projection** and **local perturbation** for large life-cycle models — surfaced *via citations* (`nishiyama_smetters` → `hasanhodzic_kotlikoff`; `hasanhodzic_kotlikoff` → `rios_rull`).
- The shared challenge is carrying the **cross-sectional distribution as part of the aggregate state** (`harenberg_ludwig_2018`: $\Phi' = H(\Phi, z, z')$).
- One paper avoids GE entirely: `pashchenko_porapakkarm_2025` solves a **partial-equilibrium** problem and aggregates by Monte-Carlo simulation rather than a price fixed point.

© Lorenz Feng
Redistribution prohibited without permission

Demo 1: Scored Retrieval and What It Shows

Top retrieved chunks (cosine):

#	score	type	source
1	0.827	citation	`nishiyama_smetters` → `hasanhodzic_kotlikoff`
3	0.767	citation	`hasanhodzic_kotlikoff` → `rios_rull`
6	0.646	element	`azinovic_yang_cole_kubler_2023 :: solution_method`
10	0.623	element	`krueger_mitman_perri_2016 :: solution_method`

© Zheng Feng • Redistribution prohibited without permission

What this shows

Citation edges top the ranking (0.83–0.77) above every element chunk (0.65–0.62) — the citation graph surfaced the best evidence; `auerbach_kotlikoff/storesletten_…` are citation-only, outside the 133 papers.

Demo 2: A Plain Research Question (the Prompt)

A question you would put to any macroeconomist — with *no mention of the corpus*:

Question. If a government runs persistent deficits and issues more bonds, does that reduce the economy's capital stock?

Retrieval silently supplies the exact accounting and mechanisms (abridged):

```
CONTEXT (retrieved from the OLG GraphRAG):
[10] abel_panageas_2023 :: market_clearing
     capital = saving net of bonds:   K_{t+1} = S_t - B_{t+1}
[8]  diamond_1965 :: causal_channel
     debt-service taxes cut young saving; internal debt competes for portfolios
     -> capital stock falls, interest rate rises.
[1]  azinovic_yang_cole_kubler_2023 :: welfare_result
     calibrated economy: government debt is never Pareto-improving; crowds out capital.
...  (10 chunks: 6 element + 4 citation, across 3 papers)
```

© Learning ● Redistribution prohibited without permission

Demo 2: The LLM's Answer (Verbatim, Abridged)

Answer — written from the augmented prompt alone

Yes — in these OLG models government debt mechanically **crowds out capital**, and the retrieved evidence pins down both the accounting and the economics:

- **The accounting is explicit.** `abel_panageas_2023` states capital-market clearing as $K_{t+1} = S_t - B_{t+1}$: for given saving, more bonds mean less capital, one-for-one.
- **The channels** (`diamond_1965`): debt service needs *taxes* that cut the young's saving (supply side); internal debt competes on the *demand* side — interest rate up, capital down.
- **Whether that is *bad* hinges on dynamic efficiency.** `samuelson_1958` / `abel_mankiw_summers_zeckhauser_1989` (cited) establish debt can *raise* welfare in dynamically inefficient economies.
- **Quantitatively**, `azinovic_yang_cole_kubler_2023` finds debt is **never Pareto-improving** in its calibration.

(c) Zhigang Feng. Redistribution prohibited without permission

Demo 2: What This Shows (Retrieval Honesty)

The answer is grounded *solely* in what retrieval pulled — so it is only as balanced as the retrieved prompt.

A one-sided pull

The **Ricardian-equivalence** counterview (debt is neutral; Barro) did *not* surface — though `barro_1974` is in the corpus, this phrasing pulled the OLG crowding-out tradition instead. That is why `--show-prompt` matters: you can see *what* grounded the answer.

© Zhigang Feng Redistribution prohibited without permission

Generalising the Recipe to Other Fields

The pilot is OLG, but the workflow is not OLG-specific. In quant-macro the field's *paradigm* is:

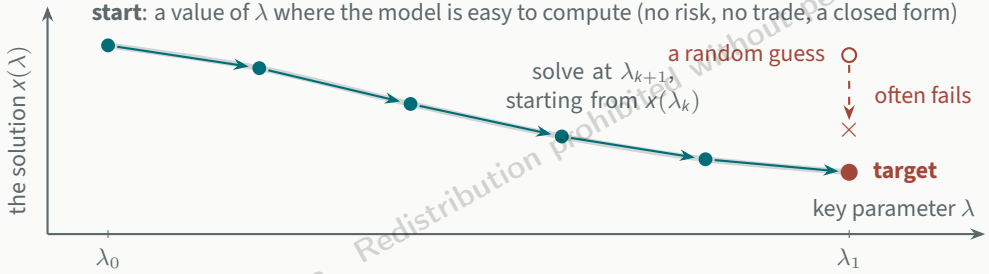
$$\textit{model} \;\rightarrow\; \textit{equilibrium definition} \;\rightarrow\; \textit{propositions \& theorems} \;\rightarrow\; \textit{calibration / estimation}$$

The $9 \rightarrow 51$ PromptEcon framework is the machine-readable form of *that* paradigm: the categories encode the universal recipe, the 51 axes are the populated slots per paper, and the typed edges encode how slots depend on one another. Reusing the recipe in another field requires re-distilling its paradigm first — the category/axis table is the deliverable.

- Other quant-macro subfields (HANK, RBC, Bewley–Aiyagari): same paradigm, different slot values — recipe carries over.
- Other social-science fields: distil that field's paradigm from its textbook synthesis, then specialise the schema.
- Empirical work: paradigm is structurally different (data design, identification, robustness) — re-derive before reuse.

© Zhigang Feng Redistribution prohibited without permission

Homotopy: Compute the Easy Model, Then Move the Key Parameters



Redistribution prohibited without permission

- **Start easy:** the value of the key parameter at which the model is easy to compute.
- **Walk the parameter** to its target in small steps: each solve starts from the last solution, and a step that fails is halved and retried.
- **Session 4's OLG lowered the borrowing limit from 0 to -0.05 this way.**

© Zhiyang Feng

When You Use AI: Five Things to Remember

plan mode
think the problem through first; start simple, then step toward the full one (homotopy)

reflections
make sure the AI has understood the problem, and that its results are verified

gates
a check fixed in advance at every step: verification that stops a mistake early

orchestration
break a complex problem into smaller tasks, and make sure they scale up

gut feeling: domain knowledge still matters — you need to understand the theory and its principles, to know when an answer is wrong

© Zhigang Feng Redistribution prohibited without permission

- **Start simple, check every step, and keep the economics in your own head.**